Distributed Multimodal Video Note-Taking Agent

1 System Overview

Build a distributed, asynchronous system that turns a YouTube video into structured study notes — summarized text plus real video frames of important diagrams/slides — and publishes the result to a Notion page. The system is driven entirely from a browser extension; the user never calls the API by hand. The backend processes jobs through a queue-backed worker pool and must respect the rate limits of every external API it calls.

1.1 Required Components

Four independently running pieces, each a separate process/container (e.g. via Docker Compose), so that killing or scaling one can be demonstrated without touching the others.

- **Browser extension**
- **API Gateway** — accepts jobs and exposes status/results. It does not perform the pipeline work itself.
- **Message broker** — durable queue, at-least-once delivery, redelivery on crash.
- **Worker pool** — ≥ 2 independent processes that pull and execute jobs.

2 Client: Browser Extension

- **Content script.** Runs inside the YouTube tab. Injects a "Take Notes" button into the player toolbar on watch pages. On click, captures the current video ID and sends it to the background script. **Must** keep working when the user navigates from one video to another.
- **Background script.** One-time Notion OAuth flow and stores the resulting token so the user isn't re-prompted on every job. Submits jobs to the gateway and relays status updates to the popup.
- **Popup UI.** The visible window the user opens by clicking the extension icon. Shows the job's progress as a human-readable sequence (Queued → Transcribing → Summarizing → Extracting Frames → Publishing → Done), and on success shows a one-click "View in Notion" link. Failures and "completed with warnings" **must** be shown clearly — raw JSON is never the end-user experience.

3 Backend Pipeline

1. **Transcript extraction** — fetch the timestamped transcript for the video. This is also where you learn the true video duration, so it's the natural place to enforce Constraint A (§7) if you didn't already reject the video at submission time.
2. **LLM synthesis** — prompt the model to produce summary bullet points and to identify the timestamps at which visually important diagrams or slides appear. For every timestamp it flags, the LLM **must** also produce a short caption explaining what is shown and why it was flagged — this caption is what gets attached to the frame in Notion, and is required by Constraint B below. If the transcript is too long for one call, chunk it and merge the partial results, keeping section order intact.
3. **Frame extraction** — seek to each identified timestamp and capture one frame directly from the video stream.
4. **Notion assembly** — upload the extracted frame images and assemble the final page (headings, bullet text, images with captions) via the Notion API.

4 Job Lifecycle

Status	Meaning
QUEUED	Accepted, not yet claimed by a worker.
TRANSCRIBING / SUMMARIZING / EXTRACTING_FRAMES / PUBLISHING	One per pipeline stage, in this order.
COMPLETED	Page published, all requested frames embedded.
COMPLETED_WITH_WARNINGS	Page published; one or more frames omitted or a non-fatal issue occurred.
FAILED	Terminated by a non-retryable error.
DEAD_LETTERED	Retry budget exhausted; routed to the DLQ for inspection.

From	To	Trigger
—	QUEUED	Job accepted and enqueued.
QUEUED	TRANSCRIBING	A worker claims the job.
TRANSCRIBING	SUMMARIZING	Transcript obtained.
SUMMARIZING	EXTRACTING_FRAMES	LLM synthesis complete.
EXTRACTING_FRAMES	PUBLISHING	Frame capture finished (0 or more frames captured).
PUBLISHING	COMPLETED / COMPLETED_WITH_WARNINGS	Page created; the with-warnings variant applies if any frame was omitted.
any non-terminal	FAILED / DEAD_LETTERED	Non-retryable error, or retry budget exhausted (§6).

5 Required REST API

POST /jobs — submit a job

```
// Request
{ "youtube_url": "https://www.youtube.com/watch?v=dQw4w9WgXcQ",
  "notion_token": "secret_...",
  "notion_parent_id": "optional-page-or-db-id" }

// Response 202 Accepted, Location: /jobs/{job_id}
{ "job_id": "uuid", "status": "QUEUED", "video_id": "dQw4w9WgXcQ" }
```

Validates the URL and returns immediately (no pipeline work happens inside this request); the job is created and enqueued, then processed asynchronously. Invalid input returns `4xx` with a short JSON error body `{"error": {"code", "message"}}`. `notion_token` is the token your background script already obtained through the one-time OAuth flow (§2) — the extension supplies it on every submission so the gateway never has to store it. If `notion_parent_id` is omitted, the job publishes to a default destination that you choose and must state in your README (e.g. a fixed page granted to the integration at setup time).

GET /jobs/{job_id} — poll status

```
{ "job_id": "uuid", "status": "EXTRACTING_FRAMES", "progress": 68,
  "video_title": "...", "duration_sec": 1147,
  "warnings": [], "error": null, "result": null }
```

Read-only and idempotent — polling **must not** mutate job state. Unknown `job_id` returns `404`.

GET /jobs/{job_id}/results — fetch the outcome

```
{ "status": "COMPLETED_WITH_WARNINGS",
  "result": { "notion_page_url": "https://notion.so/...",
              "frames_requested": 6, "frames_published": 5 },
  "warnings": [ { "code": "FRAME_EXTRACTION_FAILED", "timestamp_sec": 461 } ] }
```

Called before the job reaches a terminal state **must** return `409` (not a null result) — it is a status check, not a long-poll. On success it returns a working, absolute Notion page URL.

6 Required Failure Handling

The system **must** demonstrably handle, live, the following four failure modes. These are the scenarios you should expect a grader to trigger deliberately during your demo.

- **External rate limits (HTTP 429).** Caught and re-queued with exponential backoff (e.g. 10s, 30s, 90s) rather than surfaced as a failure while retries remain — the user should see the job waiting, not erroring. This applies to 429s from any of the three external APIs, not just the LLM provider.
- **Worker crashes.** If a worker dies mid-job (e.g. `SIGKILL` during frame extraction), the broker must detect the unacknowledged message and reassign it to a healthy worker — no job is silently lost, and no job ends up

duplicated on Notion. This requires that task acknowledgement happen only *after* a stage's work is durably complete, and that redelivering a stage is safe to repeat (idempotent), not just safe to retry.

- **Graceful degradation.** If frame extraction fails for one or more timestamps, the job still publishes the text summary (with whatever frames did succeed) rather than failing the whole job, and is marked `COMPLETED_WITH_WARNINGS`. If every frame fails, the job still publishes a text-only page.
- **Dead-letter queue.** A job that fails completely after a bounded number of attempts (e.g. 5, submitting a private/unavailable video is a natural way to trigger this) is routed to a DLQ for manual inspection instead of retried forever. The exact attempt budget and backoff schedule are your choice, but they must be bounded, configurable, and demonstrable within a reasonable time window during grading.

7 Hard Constraints

A — Video length. The system **must** correctly support videos up to 20 minutes; this is the graded baseline. Longer videos are out of scope — reject them deterministically and don't attempt to process them. You may reject at submission time. You won't lose points for failing on a video longer than 20 minutes.

B — Real frames only. Every image on the final Notion page **must** be an actual frame captured from the source video at the identified timestamp. Images **must not** be AI-generated, redrawn, or reconstructed; if extraction fails for a timestamp, that image is simply omitted, never substituted. Every inserted frame **must** be paired with an LLM-written caption explaining what it shows and why it was flagged — an image with no explanation does not satisfy this constraint.

8 Deployment & Deliverables

- Unpacked extension, loadable locally into Chrome with no build step, or with a single documented build command.
- Gateway, broker, and ≥ 2 workers, brought up together (e.g. one `docker-compose.yml`) so worker scaling and crash recovery can be demonstrated by starting/stopping individual containers or running `docker compose up --scale worker=N`.
- A short README stating the base URL, default Notion destination behaviour, and every required environment variable — at minimum an LLM API key and model name, the configured rate limit, Notion OAuth client credentials, the broker connection string, and your retry/backoff parameters. A grader should be able to get the whole stack running from the README alone, without reading your source first.
- A health-check endpoint or equivalent (e.g. `GET /healthz`) that confirms the gateway, broker, and any state store are reachable, so grading can start with a quick sanity check rather than immediately submitting a job into a half-started stack.

9 Evaluation Criteria

- **Client experience**
- **End-to-end latency** for a video near the 20-minute cap.
- **Queue resilience** — behaviour when many jobs are submitted in a short burst. "Submitting a batch of videos through the extension" means clicking Take Notes on several different videos in a row, each producing its own independent job — no dedicated batch/multi-select UI is required. The queue should drain without ever triggering a 429 from the LLM provider.
- **Fault tolerance** — recovery time and correctness after a worker is explicitly killed mid-job.
- **Correctness of Constraints A and B** on a held-out video and a half-dozen submitted videos, LLM-as-judge.

10 Left to Your Design

Deliberately unspecified, and expected to differ across submissions — be ready to justify your choices:

- Message broker technology (RabbitMQ, Redis Streams, or equivalent).
- How job state is stored and modeled, and the internal state machine used.
- How the global LLM rate limit is coordinated across workers.
- Worker topology — one worker doing all four stages vs. specialized workers per stage.
- LLM provider/model, transcript-fetching method, frame-extraction implementation.
- Extension framework/tooling and popup styling.